Transferring AppDynamics configuration and custom dashboards

From the non-prod controller **pobal-ncs-dev** to the prod controller **pobal-ncs**

Prepared for Arik Warshager, POBAL | Prepared by Konrad Wenc, Cisco AppDynamics | 10 November 2026 | Version 1.0

	Controller URL	Account name
Source (non-prod)	https://pobal-ncs-dev.saas.appdynamics.com	pobal-ncs-dev
Target (prod)	https://pobal-ncs.saas.appdynamics.com	pobal-ncs

The account names are normally the first part of the controller URL. To confirm, open the controller, click the gear icon in the top right, then **License**. The account name is shown on that page.

1. How the transfer works

You use two tools from the package, each for what it does best:

- **Config Exporter 26.7.0.0-BETA** (a small web app that runs on your laptop). Use it to **browse, inventory and download** all non-prod configuration as a backup and reference. In this beta, the screens that write configuration into another controller (*Migration* and *File Upload*) are not finished yet, so do not use them.
- **Copy-AppDConfig.ps1** (a PowerShell script). It uses the AppDynamics Configuration Import and Export REST API to **export** configuration from non-prod to a folder, and **import** that folder into prod.

The overall flow:

1. Prepare the laptop and user accounts (section 3).
2. Back up non-prod and prod with Config Exporter (section 4).
3. Export the configuration and dashboards from non-prod with the script (section 5).
4. Review the exported files and decide on name changes (section 6).
5. Import into prod with the script (section 7).
6. Check the result in prod (section 9).

What is transferred, and how

Configuration	Transferred by	Notes
Custom business transaction detection rules	Script	Custom match and exclude rules for the application.
Health rules	Script	Existing rules with the same name are kept unless you use **-Overwrite**.
Actions (email, HTTP request and so on)	Script	Email and HTTP request settings must exist in prod (section 6).
Policies	Script	Imported last, because they refer to health rules and actions by name.
Custom dashboards	Script, or the controller UI	Script: by dashboard ID. UI: export and import JSON (section 8).
Full application configuration (XML)	Script, export only	Saved as **application-backup.xml** for reference. It is never imported.
Everything else (data collectors, information points, backend detection, error detection, EUM, service endpoints and so on)	Config Exporter download, then set up in prod by hand	Use the downloaded files and CSV lists as a checklist when you configure prod.

2. What is in the package

File	Purpose
config-exporter-26.7.0.0-BETA.war	Config Exporter application.
Start-ConfigExporter.bat	Starts Config Exporter on **http://localhost:8282**. Double-click it.
Copy-AppDConfig.ps1	PowerShell script that exports and imports configuration and dashboards.
POBAL_AppD_Config_Transfer_Guide.pdf	This guide.
README.txt	Short pointer to this guide.

3. Before you start

Laptop

- Windows 10 or 11. Windows PowerShell 5.1 is built in, so nothing extra is needed for the script.
- **Java 17 or newer**, needed only for Config Exporter. Recommended: **Eclipse Temurin 21 JRE** from https://adoptium.net. During install, tick **Add to PATH**. To check, open a Command Prompt and run `java -version`.
- HTTPS access (port 443) to both controller URLs. If your network needs a proxy:
  - The script uses the Windows proxy settings automatically.
  - For Config Exporter, open **Start-ConfigExporter.bat** in Notepad and set **CE_PROXY_HOST** and **CE_PROXY_PORT** at the top.
- Unzip the package to a local folder, for example `C:\Tools\AppD-Transfer`. Then right-click each file, choose **Properties**, tick **Unblock** if it is shown, and click **OK**. Windows blocks files downloaded from email by default.

AppDynamics users

- **Non-prod (pobal-ncs-dev):** a local AppDynamics user that can view the application configuration and dashboards.
- **Prod (pobal-ncs):** a local AppDynamics user that can change application configuration and create dashboards, for example a user with the **Administrator** role.
- Both tools sign in with user name and password. Enter the user name **without** `@account`. The tools add the account name for you.

Prod application

- The target application must **already exist** in prod. It is created when the first agent registers, or you can create it under **Applications > Create Application**.
- Use the **same tier names** in prod as in non-prod where possible. Health rules and dashboards refer to tiers by name. If names differ, section 6 shows how to change them during the import.
- Write down the exact application names (they are case-sensitive). In the commands below, replace **NONPROD-APP-NAME** and **PROD-APP-NAME** with these names.

4. Back up with Config Exporter

> Do this for **both** controllers before importing anything. The prod backup lets you see and restore what was there before the transfer.

Start Config Exporter

1. Double-click **Start-ConfigExporter.bat**. A black window opens. It checks Java and starts the tool.
2. After about 20 to 30 seconds your browser opens **http://localhost:8282**. If it does not, open that address yourself.
3. Keep the black window open while you work. Close it to stop Config Exporter.

Add the two controllers

1. Go to **Controllers & Settings** and click **Add Controller**.
2. **Controller URL:** https://pobal-ncs-dev.saas.appdynamics.com **Account:** pobal-ncs-dev
3. Choose **Basic Auth**, enter the **Username** and **Password**, and click **Save**.
4. Repeat for prod: **Controller URL** https://pobal-ncs.saas.appdynamics.com, **Account** pobal-ncs.

Download the configuration

1. Open **Exporter** and select the controller, then the application.
2. Pick a configuration type from the list on the left, for example **Health Rules**, **Policies**, **Actions**, **Transaction Detection**, **Data Collectors**, **Information Points**, **Backend Detection** or **Error Detection**.
3. Click **Download** to save the configuration file, and **CSV** to save a readable list of the items.
4. Under **Dashboards**, open **Dashboards & Dash Studio** to list and download the dashboards.
5. Save everything in a folder per controller, for example `backup-nonprod` and `backup-prod-before-import`.

> **Do not use** the **Migration** page or the **File Upload** tab in this beta. They are not finished. In **Controllers & Settings > Export Controllers**, leave **Include passwords / tokens** unticked.

Config Exporter keeps your passwords in memory only and accepts connections from your own laptop only. You need to sign in again each time you start it.

5. Export from non-prod with the script

Find the dashboard IDs

In the non-prod controller, open each custom dashboard you want to copy. The ID is the number after `dashboard=` in the browser address bar, for example `...&dashboard=`**`1234`**. Write down the IDs.

Open PowerShell in the package folder

In File Explorer, open the package folder, click the address bar, type **powershell** and press Enter. Then allow scripts for this PowerShell window only:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

Run the export

```
.\Copy-AppDConfig.ps1 -Mode Export `
    -ControllerUrl https://pobal-ncs-dev.saas.appdynamics.com `
    -Account pobal-ncs-dev `
    -Application "NONPROD-APP-NAME" `
    -DashboardIds 1234,1235 `
    -Folder .\nonprod-export
```

A sign-in box opens. Enter the non-prod user name (without @pobal-ncs-dev) and password. The script then shows one line per item, for example:

```
  [OK] AppBackup -> C:\Tools\AppD-Transfer\nonprod-export\application-backup.xml
  [OK] TransactionDetection -> ...\transaction-detection-custom.xml
  [OK] HealthRules -> ...\health-rules.xml
  [OK] Actions -> ...\actions.json
  [OK] Policies -> ...\policies.json
  [OK] Dashboard 1234 -> ...\dashboards\dashboard-1234.json
Done: 7 OK, 0 skipped, 0 failed.
```

If you have no custom dashboards, leave out **-DashboardIds**. The script skips dashboards and exports the rest.

6. Review the files before importing

Open the files in the export folder in Notepad and check the following:

- **Application and tier names.** If prod uses different names, the script can change them during the import with **-Replace** (section 7). The replacement is exact text and case-sensitive. The files on disk are not changed.
- **Health rules scoped to specific nodes or servers.** Non-prod node and server names do not exist in prod. Change these rules to tier scope, or fix their scope in prod after the import.
- **Email actions.** Check the recipient addresses are right for prod. Prod also needs email and SMS settings under **Settings > Administration > Email / SMS**.
- **HTTP request actions.** The HTTP request templates they use must exist in prod first (**Alert & Respond > HTTP Request Templates**). Create them there, or recreate the actions by hand.
- **Empty transaction-detection-custom.xml.** Either non-prod has no custom detection rules, or they use the newer rule model. Check the **Config 2.0 Rules** and **Config 2.0 Scopes** downloads from Config Exporter and recreate those rules in prod by hand.

7. Import into prod with the script

> This step **writes into the prod controller**. Make sure you have the prod backup from section 4 first. You can also back up prod with the script by running the export command against prod: **-ControllerUrl https://pobal-ncs.saas.appdynamics.com -Account pobal-ncs -Application "PROD-APP-NAME" -Folder .\prod-backup**.

Run the import from the same PowerShell window. If you opened a new window, run the `Set-ExecutionPolicy` line from section 5 again.

```
.\Copy-AppDConfig.ps1 -Mode Import `
    -ControllerUrl https://pobal-ncs.saas.appdynamics.com `
    -Account pobal-ncs `
    -Application "PROD-APP-NAME" `
    -Folder .\nonprod-export `
    -Replace @{ 'NONPROD-APP-NAME' = 'PROD-APP-NAME' }
```

1. Sign in with the **prod** user name (without @pobal-ncs) and password.
2. The script shows the target controller, the application and any text replacements. Type **YES** (in capitals) to continue.
3. It imports in this order: transaction detection rules, health rules, actions, policies, dashboards. Each line shows OK, SKIPPED or FAILED, with the controller's reply.

Useful options

Option	What it does
-Replace @{ 'a' = 'b'; 'c' = 'd' }	Changes text in every file before upload, for example application or tier names. Separate several pairs with a semicolon.
-Overwrite	Replaces health rules and transaction detection rules that already exist in prod with the same name. Without it, existing rules are kept.
-Include HealthRules,Policies	Runs only the listed parts. Choose from TransactionDetection, HealthRules, Actions, Policies, Dashboards and AppBackup (AppBackup applies to export only). Useful for re-running one failed step.
-Folder .\name	Folder to write to (export) or read from (import).

> If a step fails, read its message, fix the cause, and re-run only that step with **-Include**. Avoid re-running actions and policies that already imported, because that can create duplicates.

8. Dashboards: alternative using the controller UI

You can also copy custom dashboards one by one in the controller, without the script:

1. In **non-prod**, go to **Dashboards & Reports**, select the dashboard and choose **Export**. A JSON file is downloaded.
2. If the application or tier names differ in prod, open the JSON in Notepad and use **Edit > Replace** to change them.
3. In **prod**, go to **Dashboards & Reports**, click **Import** and choose the JSON file.

- Widgets that refer to applications, tiers, nodes or business transactions that do not exist in prod show no data. Edit those widgets after the import.

- **Dash Studio** dashboards are not covered by the script. Use Dash Studio's own export and import if your controller has it, or recreate the dashboard in prod.

9. Check the result in prod

Check	Where in the prod controller
Custom detection rules are present and enabled	Application > **Configuration > Instrumentation > Transaction Detection**
Health rules are present, enabled and point at the right tiers	Application > **Alert & Respond > Health Rules**
Actions have the right recipients and templates	Application > **Alert & Respond > Actions**
Policies link the right health rules to the right actions	Application > **Alert & Respond > Policies**
Dashboards open and show data	**Dashboards & Reports**
Items that are set up by hand (data collectors, information points and so on) match non-prod	Compare with the Config Exporter CSV lists from section 4

When you are done, delete exported files you no longer need. They contain configuration details and email addresses.

10. Troubleshooting

Problem	What to do
Config Exporter: *Java was not found* or *needs Java 17 or newer*	Install Eclipse Temurin 21 JRE with **Add to PATH** ticked. Open a new window and try again.
Config Exporter: *Port 8282 is already in use*	It is already running in another window. Close it, or change **CE_PORT** at the top of the .bat file.
PowerShell: *running scripts is disabled* or *is not digitally signed*	Run the `Set-ExecutionPolicy -Scope Process ...` line from section 5, and unblock the files (section 3).
HTTP 401 Unauthorized	Wrong user name, password or account name. Enter the user name without @account.
HTTP 403 Forbidden	The user's role does not allow this. Use a user with more rights on that controller.
Application ... not found	The name must match exactly. The message lists the applications the script can see.
Policies fail to import	The health rules or actions they refer to are missing in prod. Import those first, then re-run with **-Include Policies**.
Connection errors or timeouts	Check the proxy. The script uses the Windows proxy settings, and Config Exporter uses the proxy set in the .bat file.
Config Exporter problem to report	Send **config-exporter.log** from the **logs** folder next to the .war file.

Questions: contact Konrad Wenc (Cisco AppDynamics).